

HeroesAI

Taktyczna gra bitewna w stylu *Heroes of Might and Magic III*
Dokumentacja końcowa

Łukasz Szydlik Dominik Śledziewski

Czerwiec 2026

Abstract

HeroesAI to napisana w języku C++ taktyczna gra bitewna inspirowana ekranem walki z klasycznej gry *Heroes of Might and Magic III*. Dwóch bohaterów toczy pojedynek na heksagonalnym polu bitwy, dowodząc oddziałami stworzeń i korzystając z księgi zaklęć. Stroną może sterować człowiek albo jeden z trzech botów, w tym *minimax* z odcinaniem alfa-beta. Dokument opisuje zrealizowaną funkcjonalność, architekturę aplikacji, użyte techniki i biblioteki, sposób testowania oraz krytyczne podsumowanie napotkanych problemów.

Spis treści

1	Temat projektu	1
2	Zrealizowana funkcjonalność	2
2.1	Co zostało zaimplementowane	2
2.2	Czego nie zrealizowano / uproszczenia	2
2.3	Odstępstwa od pierwotnego planu	2
3	Architektura aplikacji	3
3.1	Główne klasy i ich rola	3
4	Użyte techniki, wzorce i biblioteki	4
4.1	Wzorce projektowe i mechanizmy języka	4
4.2	Biblioteki zewnętrzne	4
5	Dokumentacja użytkownika	5
5.1	Wymagania i kompilacja	5
5.2	Uruchomienie	5
5.3	Konfiguracja bitwy	5
6	Testy i jakość	5
7	Pracochłonność – plan a wykonanie	6
8	Napotkane problemy i krytyczne podsumowanie	7

1 Temat projektu

Celem projektu było zaimplementowanie samodzielnej, taktycznej gry bitewnej osadzonej na heksagonalnej planszy, wzorowanej na fazie walki znanej z gier z serii *Heroes of Might and Magic*. Pojedyncza bitwa to starcie dwóch armii, z których każda składa się z maksymalnie siedmiu *oddziałów* (stosów identycznych jednostek) dowodzonych przez bohatera dysponującego punktami

many i księgą zaklęć. Gracze wykonują akcje naprzemiennie, w kolejności wyznaczonej przez inicjatywę (szybkość) jednostek.

Zadanie celowo ograniczono do samej *fazy bitwy* (bez warstwy strategicznej, mapy przygody czy ekonomii), tak aby mogło zostać zrealizowane w zaplanowanym czasie, a jednocześnie pozostawało nietrywialne algorytmicznie. Kluczowym, ambitnym elementem jest sztuczna inteligencja: oprócz typowych botów zaimplementowano przeciwnika opartego o algorytm *minimax* z odcinaniem alfa-beta, działającego na niezależnej kopii stanu gry.

2 Zrealizowana funkcjonalność

2.1 Co zostało zaimplementowane

- **Heksagonalne pole bitwy** 15×11 w układzie współrzędnych sześciennych (*cube coordinates*) wraz z wyszukiwaniem ścieżki.
- **System tur i rund** z inicjatywą: szybsze oddziały działają pierwsze; akcje **Czekaj** (wait) oraz **Obrona** (defend); kolejka jednostek odłożonych po komendzie czekania.
- **Walka wręcz i dystansowa** z kontratakiem, ograniczoną amunicją strzelców, karą za strzał w zwarcie oraz losowymi rzutami na morale, które mogą przyznać dodatkową turę.
- **Księga 13 zaklęć** (Magic Arrow, Lightning Bolt, Ice Bolt, Cure, Curse, Bless, Haste, Slow, Blind, Bloodlust, Stone Skin, Shield, Disrupting Ray) oraz system tymczasowych modyfikatorów (*buffów*) wpływających na statystyki jednostek.
- **Premie bohatera** (atak, obrona, moc, wiedza) wpływające na oddziały oraz pulę many ($= 10 \times$ wiedza).
- **Roster jednostek sterowany danymi** (frakcje Castle, Inferno, Necropolis) wczytywany z pliku JSON (`assets/units.json`).
- **Trzy strategie SI** wybierane niezależnie dla każdej ze stron: **random**, **easy** (heurystyka priorytetów) oraz **minimax** (przeszukiwanie alfa-beta z porządkowaniem ruchów i przycinaniem gałęzi).
- **Animacje sprite'ów** w oryginalnym, binarnym formacie `.def`, samodzielnie parsowanym i renderowanym przez SFML.
- **Trzy sceny aplikacji** (menu główne, dobór armii, bitwa) zarządzane przez **SceneManager**, z konfiguracją meczu w pliku `settings.cfg`.
- **Ekran końca bitwy**: po wykryciu, że jedna ze stron straciła wszystkie oddziały, na środku ekranu pojawia się napis „GAME OVER” wraz z informacją, który gracz wygrał; po 5 sekundach następuje automatyczny powrót do menu głównego.

2.2 Czego nie zrealizowano / uproszczenia

- Brak warstwy strategicznej (mapa przygody, rozwój bohatera między bitwami) – zgodnie z założonym zawężeniem tematu.
- Jednostki dwuheksowe są modelowane w danych (pole `size`), lecz renderowane i traktowane w logice ruchu jako jednoheksowe.
- Zrezygnowano z zaplanowanej w dokumentacji wstępnej porównania skuteczności SI z oryginałem (bitwy budowane w edytorze map gry *Heroes III*) zrealizowano jedynie w ograniczonym, nieformalnym zakresie.

2.3 Odstępstwa od pierwotnego planu

W trakcie realizacji kilka decyzji projektowych odbiegło od dokumentacji wstępnej – w każdym przypadku na rzecz prostszego lub bogatszego rozwiązania:

- **Biblioteka Boost** (planowana) okazała się zbędna: do parsowania danych użyto `nlohmann/json`, a do zarządzania zasobami i kolekcjami – biblioteki standardowej (sprytne wskaźniki, kontenery, algorytmy). Pozwoliło to uniknąć ciężkiej zależności.

- **Algorytm A*** (planowany) zastąpiono przeszukiwaniem wszere (BFS). Na małej, jednorodnej siatce heksów bez wag krawędzi BFS daje identyczny wynik mniejszym kosztem implementacyjnym.
- **System zaklęć i buffów** (13 czarów, premie bohatera) jest rozszerzeniem wykraczającym poza zakres planu wstępnego – znacząco zwiększył wartość taktyczną gry, ale i pracochłonność modułu mechaniki.

3 Architektura aplikacji

Aplikacja zbudowana jest według wzorca **Model–View–Presenter (MVP)** i podzielona na cztery współpracujące moduły (katalogi w `src/`), o jasno rozdzielonych odpowiedzialnościach i kierunku zależności skierowanym „do wewnątrz” – ku modelowi:

models

Encje dziedziny i czyste reguły danych: `Unit` (oraz `RangeUnit`), `Hero`, `Army`, `Board`, `Hex`, `Buff`, `Spell`. Moduł nie zależy od SFML.

core

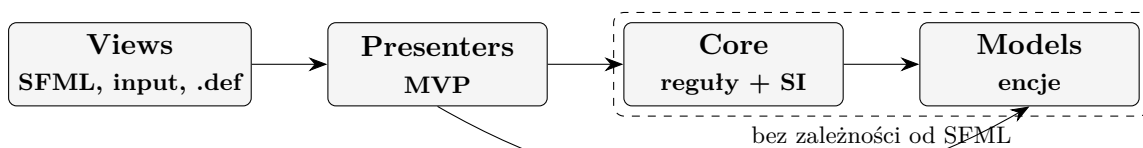
Logika gry i SI: `GameManager` (fasada bitwy), `RoundManager` (kolejność tur), `ActionManager` (ruch, atak, pathfinding), `SpellResolver`, `ActionGenerator` oraz boty (`IBot` i implementacje). Tu również sceny aplikacji.

presenters

Pośrednicy MVP tłumaczący zdarzenia interfejsu na komendy modelu i przygotowujący dane do renderowania.

views

Renderowanie SFML, parsowanie sprite’ów `.def`, kontroler animacji oraz obsługa wejścia. Zna interfejsy widoków (`IBattleView` itd.), nie zna szczegółów logiki.



Biblioteki zewnętrzne: **SFML 3.0.1** (graphics/window/system/audio), **nlohmann/json 3.11.3** (dane), **GoogleTest** (testy)

Rysunek 1: Warstwy aplikacji i kierunek zależności (MVP).

3.1 Główne klasy i ich rola

GameManager

Fasada jednej bitwy. Agreguje planszę, bohaterów, kolejkę rund i menedżera akcji. Udostępnia prezeneterowi i botom zapytania (aktywna jednostka, legalne ruchy/ataki) oraz komendy (ruch, atak, czekanie, obrona). Metoda `clone()` tworzy głęboką, niezależną kopię stanu na potrzeby przeszukiwania SI.

RoundManager

Wyznacza kolejność działania jednostek w rundzie (inicjatywa), obsługuje czekanie i odtwarzanie stanu kolejki (dla SI).

ActionManager

Bezstanowy „solver” pojedynczej akcji: oblicza osiągalne hekisy (BFS), cele ataku, wykonuje pathfinding i aplikuje skutki na planszy oraz jednostkach.

Unit / RangeUnit

Stos identycznych stworzeń. Trzyma statystyki bazowe i efektywne (po buffach), pozy-

cję, listę buffów oraz reguły otrzymywania obrażeń. `RangeUnit` dodaje amunicję i pociski. Polimorficzne `clone()` zachowuje typ dynamiczny.

IBot + implementacje

Wzorzec *strategia*: wspólny kontrakt wyboru akcji. `RandomBotService`, `EasyBotService` (priorytety) i `MinimaxBotService` (alfa-beta) są wymienne.

ActionGenerator / ActionCommand

Generuje pełny zbiór legalnych akcji jako dyskretne, oceniane obiekty-komendy – wspólne zarówno dla botów, jak i dla wizualizacji.

SceneManager / IScene

Cykl życia scen aplikacji (menu → dobór armii → bitwa) na jednym oknie SFML.

DefParser / DefManager / AnimationController

Dekodowanie binarnego formatu `.def` (paleta, grupy klatek) i sterowanie maszyną stanów animacji jednostki.

4 Użyte techniki, wzorce i biblioteki

4.1 Wzorce projektowe i mechanizmy języka

- **Model–View–Presenter** – rozdzielenie logiki od renderowania; model nie zależy od SFML, co umożliwia testowanie bez interfejsu.
- **Strategia** (`IBot`) – wymienne algorytmy decyzyjne SI.
- **Fasada** (`GameManager`) – jeden spójny interfejs do złożonego podsystemu bitwy.
- **Fabryka** (`UnitFactory`, `SpellRegistry`) – tworzenie jednostek i zaklęć na podstawie danych/identyfikatorów.
- **Komenda** (`ActionCommand`) – akcja jako oceniany obiekt.
- **Maszyna stanów** (`AnimationController`, sceny przez `IScene`).
- **Hierarchie klas i polimorfizm** – `Unit/RangeUnit` z wirtualnym `clone()`; interfejsy widoków i scen.
- **Sprytne wskaźniki** – `shared_ptr` jako właściciel jednostek, `weak_ptr` w `Hex` (uniknięcie cykli), `unique_ptr` dla scen i kopii stanu SI.
- **Obsługa błędów wyjątkami** – np. niepoprawne współrzędne heksu, brak pliku zasobu, błąd parsowania.
- **STL oraz C++ w standardzie 23** – kontenery, algorytmy, `std::erase_if`, `std::optional`, wyrażenia lambda jako modyfikatory statystyk w `Buff`.

4.2 Biblioteki zewnętrzne

Wszystkie zależności są pobierane i budowane lokalnie przez CMake (`FetchContent`) – nie wymagają instalacji systemowej i nie ingerują w katalogi systemu operacyjnego:

Biblioteka	Wersja	Zastosowanie
SFML	3.0.1	grafika, okno, system, audio
nlohmann/json	3.11.3	parsowanie danych jednostek i konfiguracji
GoogleTest	1.x	framework testów jednostkowych

Planowana pierwotnie biblioteka **Boost** ostatecznie nie była potrzebna (jej rolę przejęły `nlohmann/json` oraz biblioteka standardowa), dzięki czemu projekt korzysta wyłącznie z pakietów dostępnych bez systemowej instalacji. Żadna z bibliotek nie realizuje całego zadania – cała logika gry oraz parser formatu `.def` są własnym kodem. Użycie bibliotek jest jawnie zaznaczone w kodzie (dyrektywy `#include`) i w niniejszej dokumentacji.

5 Dokumentacja użytkownika

5.1 Wymagania i kompilacja

Wymagany jest **CMake** ≥ 3.20 oraz kompilator wspierający standard C++23. Pozostałe zależności (SFML, nlohmann/json, GoogleTest) pobiera i buduje automatycznie CMake za pomocą **FetchContent** – nie wymagają one ręcznej instalacji. Projekt budowano i uruchamiano zarówno na Linuksie, jak i na Windowsie.

Linux (testowano: Ubuntu 24.04 LTS, g++ 13). Należy najpierw zainstalować systemowe nagłówki deweloperskie backendów SFML:

```
sudo apt update
sudo apt install -y build-essential cmake git \
    libfreetype-dev libxrandr-dev libxcursor-dev libxi-dev \
    libudev-dev libopenal-dev libflac-dev libvorbis-dev libgl1-mesa-dev
```

Konfiguracja i budowanie (generator jednokonfiguracyjny, np. Make/Ninja – rodzaj kompilacji ustala się na etapie konfiguracji):

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

Windows (testowano: VS Code). Wystarczy CMake oraz zestaw narzędzi kompilatora C++ (MSVC z Visual Studio Build Tools lub MinGW); backendy SFML korzystają z bibliotek systemowych Windows, więc nie trzeba instalować dodatkowych pakietów. Konfiguracja i budowanie (generator wielokonfiguracyjny – rodzaj kompilacji podaje się dopiero przy budowaniu):

```
cmake -S . -B build
cmake --build build --config Release
```

5.2 Uruchomienie

Program należy uruchamiać z katalogu głównego projektu, aby odnalazł katalog **assets/** oraz plik **settings.cfg**:

```
# Linux
./build/HeroesAI
# Windows
./build/Release/HeroesAI.exe
```

5.3 Konfiguracja bitwy

Plik **settings.cfg** (format JSON) pozwala zmienić ustawienia bez rekompilacji. Najważniejsze pola: **player.blue/player.red** (human/random/easy/minimax), **player.depth** (głębokość minimaksa), **heroes.*** (statystyki bohaterów) oraz **left_army/right_army** (składy armii: nazwa jednostki z **units.json** i liczebność).

6 Testy i jakość

Testy jednostkowe napisano przy użyciu biblioteki **GoogleTest** oraz zintegrowano je z systemem CTest (z automatycznym wykrywaniem przypadków testowych). Pokrywają one warstwę modelu i logiki gry: konstrukcję planszy, armie i bohaterów, jednostki, system buffów oraz premie bohatera, wczytywanie rosteru jednostek z danych (**UnitFactory**), kolejność tur, reguły ruchu i walki,

generator akcji, a także niezależność kopii stanu i zachowanie bota minimax oraz wykrywanie końca bitwy.

Metryka	Wartość
Liczba linii kodu (źródła produkcyjne)	13 072
Liczba linii kodu (testy)	1 147
Liczba linii kodu (łącznie)	14 219
Liczba plików źródłowych (.cc/.h)	77
Liczba testów jednostkowych	64 (14 zestawów)
Pokrycie liniowe warstwy logiki (core+models)	69 %

Pokrycie zmierzono narzędziem `gcovr` na kompilacji instrumentowanej (`--coverage`). Warstwa graficzna (`views`, `presenters`) nie jest objęta testami automatycznymi – ze względu na zależność od kontekstu graficznego SFML weryfikowano ją ręcznie (testy manualne). Uruchomienie testów:

```
# Linux
ctest --test-dir build --output-on-failure # lub: ./build/HeroesAITests
# Windows
ctest --test-dir build -C Release --output-on-failure # lub: ./build/Release/
HeroesAITests.exe
```

7 Pracochłonność – plan a wykonanie

Poniższa tabela zestawia szacunki z dokumentacji wstępnej (kolumna „Plan”) z rzeczywistym czasem pracy („Wyk.”), zachowując ten sam podział na zadania co dokument wstępny. Zgodnie z zasadami przedmiotu testy automatyczne są nieodłączną częścią implementacji każdej funkcjonalności i nie są wykazywane osobno.

Zadanie (wg dokumentacji wstępnej)	Plan [h]	Wyk. [h]
1. Środowisko i architektura bazowa (CMake, biblioteki, pętla gry, sceny)	10	12
2. Plansza i geometria heksagonalna (współrzędne, wyszukiwanie ścieżki)	12	12
3. Modele danych i mechanika bitwy (jednostki, walka, kontrataki, inicjatywa)	39	48
4. Sztuczna inteligencja (klonowanie stanu, heurystyka, minimax alfa-beta)	35	32
5. Warstwa graficzna i interfejs (render heksów/sprite’ów, wejście, menu)	36	42
6. Porównanie z oryginałem (Heroes III) – scenariusze i analiza skuteczności	10	4
Suma (zespół)	142	150
Na osobę (średnio)	71	75

Porównanie z planem wskazuje trzy istotne rozbieżności. Po pierwsze, **warstwa graficzna** (poz. 5) znacząco przekroczyła szacunek – głównie przez konieczność napisania własnego parsera nieudokumentowanego formatu `.def` oraz migrację na zrywające zgodność API SFML 3, czego plan nie przewidywał. Po drugie, **mechanika bitwy** (poz. 3) urosła, ponieważ dodano niezaplanowany

system 13 zakłęg i buffów. Po trzecie, **porównanie z oryginałem** (poz. 6) zrealizowano w okrojonym zakresie, oddając czas na dopracowanie rdzenia rozgrywki. Pozostałe zadania zamknęły się blisko planu, co potwierdza zasadniczą poprawność wstępnej dekompozycji.

8 Napotkane problemy i krytyczne podsumowanie

Inżynieria wsteczna formatu `.def`

Oryginalny, binarny format animacji (paleta, grupy klatek, przesunięcia) jest nieudokumentowany. Jego poprawne sparsowanie zajęło więcej czasu niż zakładano; błędne założenia co do rozmiarów płótna powodowały artefakty wizualne.

Układy współrzędnych

Pogodzenie współrzędnych sześciennych heksów z pozycjami pikselowymi oraz skalowanie „letterbox” okna było źródłem błędów geometrycznych.

Wydażność przeszukiwania

Głębokość 4 na pełnych, siedmiooddziałowych armiach jest kosztowna (kopia stanu na węzeł). Zastosowano budżet węzłów oraz agresywne przycinanie gałęzi; przy odczuwalnych zacięciach można obniżyć `depth` do 3.

Zarządzanie zasobami

Współdzielona własność jednostek (`shared_ptr`) i słabe referencje na heksach (`weak_ptr`) chronią przed cyklami; głęboka kopia `clone()` zapewnia niezależność symulacji SI od stanu gry.

Wnioski. Największą wartością projektu okazało się ściśle oddzielenie logiki od warstwy graficznej – umożliwiło ono pokrycie reguł gry testami jednostkowymi i bezpieczne „klonowanie” stanu, na którym opiera się przeszukiwanie alfa-beta bota minimax. Gdybyśmy zaczęli od nowa, trafniej oszacowali koszt napisania własnego parsera nieudokumentowanego formatu `.def` i migracji na zrywające zgodność API SFML 3 – to one, a nie sama logika gry, okazały się największym nieprzewidzianym obciążeniem prac.